

Fecha de entrega: 23:59 horas del 4 de junio de 2023

1. Objetivo

- Comprender los requerimientos de una versión reducida de la arquitectura MIPS de 32 bits para realizar su implementación en forma de procesador monociclo (ruta de datos y unidad de control).
- Codificar, ensamblar y simular un programa de prueba para verificar el comportamiento correcto del procesador implementado.
- Emplear herramientas de software para el diseño y la simulación de computadores digitales.

2. Descripción

En esta práctica cada equipo realizará una implementación como procesador monociclo de una versión reducida de la arquitectura MIPS32 en la herramienta **Logisim Evolution**. Partiendo de la implementación básica mostrada en la Figura 1, cada equipo de trabajo deberá extender el repertorio de instrucciones soportadas en hardware. Una vez implementadas la ruta de datos y la unidad de control, el procesador será puesto a prueba para verificar el correcto funcionamiento de todas las instrucciones incluidas, y usado para resolver un problema en particular asignado a cada equipo de trabajo.

El procesador debe estar en capacidad de ejecutar las siguientes instrucciones: **load word, store word, add, sub, and, or, nor, set-on-less-than, branch on equal** y **jump**, siguiendo una organización como la que se muestra en la Figura 1. Luego, cada equipo de trabajo debe agregar las instrucciones **jump-and-link** y **jump-register**, y dos más como se explica en la Sección 3 de esta guía. Los formatos de instrucción con sus códigos de operación, identificadores de registros y valores del campo función del formato R se deben ajustar a los definidos en el documento **MIPS Reference Data**. El control de la ruta de datos se debe realizar con los módulos “Control” y “ALU control” incluidos en la Figura 1, aplicando técnicas de diseño combinacional estudiadas previamente en el curso.

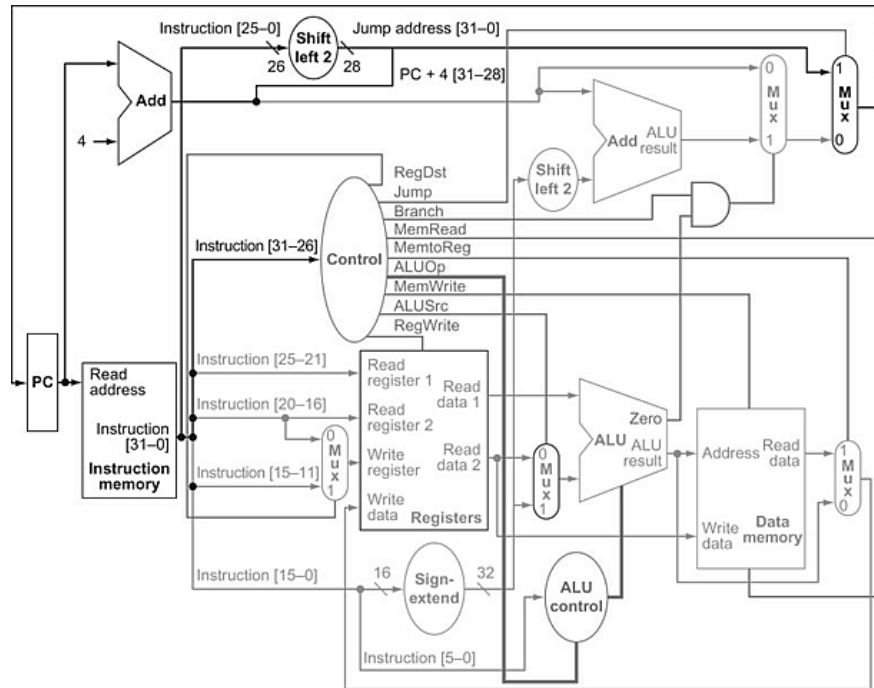


Figura 1. Implementación monociclo de la arquitectura MIPS capaz de ejecutar instrucciones *lw*, *sw*, *add*, *sub*, *and*, *or*, *nor*, *slt*, *beq* y *jump*.

El diseño del banco de registros se debe realizar siguiendo un estilo jerárquico, partiendo de un registro de 32 bits disponible en la biblioteca de componentes de **Logisim Evolution**, que será instanciado tantas veces como sea necesario para conformar el banco. Los puertos de lectura y escritura pueden ser implementados usando decodificadores y multiplexores disponibles en la misma biblioteca. El banco de registros debe cumplir con las siguientes condiciones: (i) la lectura de los registros debe ser asíncrona, mientras que la escritura debe ser síncrona; y (ii) de acuerdo con la arquitectura del conjunto de instrucciones MIPS de 32 bits, el registro **\$zero** siempre debe entregar el valor cero cuando sea accedido en modo de lectura, y su valor no puede ser modificado mediante una operación de escritura. La biblioteca de componentes de **Logisim Evolution** podrá ser empleada con total libertad para el diseño de los demás elementos del procesador.

Durante el proceso de diseño e implementación del procesador pueden presentarse diversas situaciones ante las cuales cada equipo debe tomar decisiones. En todos los casos, las decisiones de diseño deben estar ampliamente explicadas y justificadas.

3. Asignación de instrucciones adicionales y programas de prueba

A cada equipo de trabajo le corresponderá agregar dos instrucciones al procesador, y luego emplearlo para la ejecución de un programa que resuelva el problema que le es asignado. Los requisitos a los que se debe ajustar la codificación en ensamblador son los siguientes:

- Programación basada en el uso de procedimientos: el código que resuelve el problema debe ser estructurado, de manera que se tengan que emplear las instrucciones **jal** y **jr**.
- Seguir la convención para el uso de registros MIPS
- Se valorará la creatividad y elegancia en la programación.

Las dos instrucciones por agregar y el problema para cada equipo se calcula como (*Número del equipo* % 5), siendo las asignaciones las siguientes.

Asignación #0	
<i>Instrucciones:</i> Add immediate y Load halfword .	<i>Problema:</i> el programa debe tomar una matriz de 5×5 números enteros con valores entre -100 y 100, y determinar cuáles son el mayor y el menor número almacenado en cada fila y en cada columna de la matriz.
Asignación #1	
<i>Instrucciones:</i> Shift right logical y Load byte .	<i>Problema:</i> el programa debe tomar un vector de veinticinco números enteros que contiene valores entre -100 y 100, y determinar cuáles son el mayor y el menor número almacenados entre los elementos de índice par e impar.
Asignación #2	
<i>Instrucciones:</i> Shift left logical y Load upper immediate .	<i>Problema:</i> el programa debe tomar un vector de veinticinco números enteros que contiene valores entre -100 y 100, y ordenarlo de manera ascendente, indicando cuántos elementos son negativos y cuántos son positivos.
Asignación #3	
<i>Instrucciones:</i> Load halfword unsigned y And immediate .	<i>Problema:</i> el programa debe tomar un vector de veinticinco números enteros que contiene valores entre -100 y 100, y ordenarlo de manera descendente, indicando cuántos elementos son pares y cuántos son impares.
Asignación #4	
<i>Instrucciones:</i> Load byte unsigned y Branch on not equal .	<i>Problema:</i> el programa debe tomar un vector de veinticinco números enteros que contiene valores entre -100 y 100, y determinar cuáles son el mayor y el menor número almacenados, y cuántas veces se repite cada uno.

4. Informe

Cada equipo de trabajo debe realizar un informe escrito que incluya una descripción completa del proceso de diseño del procesador monociclo (ruta de datos y unidades de control), sus esquemáticos y la explicación y justificación de las decisiones de diseño tomadas durante el desarrollo de la práctica. Asimismo, debe evidenciar el proceso de verificación de la ejecución correcta de todas las instrucciones que soporta el procesador. También debe incluir una descripción de alto nivel (mediante pseudocódigo, diagrama de flujo, etc.) del programa implementado que da solución al problema asignado, suministrando su código fuente en ensamblador y el código de máquina equivalente. El informe debe contener resultados de ejecución, y las observaciones y conclusiones pertinentes del trabajo. Este documento debe estar en formato **PDF** y ser subido a la plataforma antes del cierre del plazo de entrega, con los archivos de diseño en **Logisim Evolution** como adjuntos. El informe tiene un peso del 30% en la calificación global de la práctica.

5. Sustentación

Ambos miembros del equipo de trabajo deben demostrar un dominio completo del desarrollo de la práctica, mediante un video explicativo con una duración máxima de ocho minutos, en el que intervengan de manera equitativa. La entrega del video se debe realizar como un enlace de acceso al mismo incluido en el informe, y no como un archivo adjunto. Tanto el profesor como el monitor podrán citar a algunos equipos de trabajo para que respondan preguntas que permitan aclarar o ampliar la información suministrada en el video con la sustentación. La sustentación tiene un peso del 70% en la calificación global de la práctica.